

Computer Vision – 6DOF Pose Estimation from an RGB camera – Lab 1

Contents

Summary	2
Before starting: Prepare your MATLAB workspace	2
Step 1: Visualizing the 3D points and the corresponding 2D points on the RGB image	2
Step 2: Compute the projection matrix P using DLT	3
A) Define the linear System	3
B) Solve the optimization problem	3
C) Check the validity of your projection matrix	4
Step 3: Determine the Pose T (determine the intrinsic matrix K, the rotation R, the translation t) ...	4

Summary

In this lab session, the goal is to implement the Dynamic Linear Transformations (DLT) algorithm to compute the pose of an object from a single RGB image. A partial 3D model of the object is given as well as its 2D correspondences in the 2D image. The DLT algorithm has been introduced in Lecture 7 (see Slides 36-42).

Before starting: Prepare your MATLAB workspace

1. Download the zip folder using this link:
<https://dropit.uni.lu/invitations?share=2ca1611ba73eed9f4d0>
2. Extract the zip folder and set up your workspace as the created folder
3. The folder should contain the following file:
 - a) **3D2Dpoints.mat** contains the following variables: X - the partial coordinates of the 3D object model (cube), x - their corresponding points in 2D, *startind* and *enind* - the respective start and end indices of the object edges.
 - b) **cubeRGB.JPG** contains the following RGB image showing the object (cube). The goal is to determine the pose of the visualized cube.



- c) **rq.m** is a MATLAB function that allows realizing RQ factorization.
- d) **main.m** is an empty MATLAB file. You will develop your algorithm in this file.

Step 1: Visualizing the 3D points and the corresponding 2D points on the RGB image

- Visualize on the same figure the RGB image **cubeRGB.JPG** and the 2D keypoints $x\{1, I\}$.
- Visualize on a second figure the 3D keypoints of the incomplete cube using the variable X .

- Visualize also on the second figure the edges forming the incomplete cube using the variables *startind* and *enind*.
- Report all the obtained visualization and the code in the report.

Useful MATLAB commands:

```
>>load('name_file.mat')
% load the data saved in a matfile
>> imshow('image.JPG')
% show an image on a figure
>> scatter(X,Y)
% draw 2D points
>> scatter3(X,Y,Z)
% draw 3D points
>> plot3(V1,V2,V3,'b')
% draw 3D lines with V1,V2 and V3 a set of 3D vectors
>> hold on
% draw on the same figure
>> figure
% define a new figure while keeping the previous figure
>> M=X'
% Transpose of X
```

Step 2: Compute the projection matrix P using DLT

A) Define the linear System

- In the report, explain how the correspondences between x and X can be used to obtain the following linear system $M\mathbf{v} = \mathbf{0}$. Define M and $\mathbf{v}$. What should be the size of M and $\mathbf{v}$?
- In your code, define M and check if you obtain the correct size? Report the code and the output.
- In the report, explain how the linear system $M\mathbf{v} = \mathbf{0}$ can be solved by finding the solution of the following optimization problem $\min_{\|\mathbf{v}\|=1} \|M\mathbf{v}\|^2$. What will contain the solution $\mathbf{v}$?

B) Solve the optimization problem

- In the report, explain how to solve the defined optimization problem
- Using MATLAB, solve the defined optimization problem. Report the obtained results (outputs). Compute the projection matrix P .

Useful MATLAB commands:

```
>> [u, s, v] = svd(M);  
% Singular Value decomposition  
P = reshape(vector(size(vector), a, b);  
% Reshape a vector as a matrix with a the number of lines and b  
the number of columns
```

Some useful information related to solving a linear problem using SVD:

To solve the defined optimization problem $M\mathbf{v} = \mathbf{0}$, the SVD can be used as follows:

$$\text{SVD}(M) = U^T A V$$

The solutions to the optimization problem $\mathbf{v}^$ will correspond to the last column of V .*

C) Check the validity of your projection matrix

- In the code, reproject the 3D points using the obtained matrix P and show that the obtained 2D points on the 2D image cubeRGB.JPG (use two different colors for the original points and the projected points). Compute the mean square error. Report the code and the outputs in the report.
- Interpret the obtained results.

Useful MATLAB commands:

```
>> scatter(points_proj(1,:), points_proj(2,:), 'r');  
% Show points in red color ('g' for green color instead of 'r')
```

Step 3: Determine the Pose T (determine the intrinsic matrix K , the rotation R , the translation t)

Some useful information related to determining the pose using RQ factorization:

The projection matrix P can be written as follows:

$$P = K[R|t] = K[R|-RC] = [KR|-KRC] \text{ with } C \text{ the camera center (which means that } PC = 0)$$

Let us assume that $M_1 = KR$, then

$$P = [M_1 | -M_1 C]$$

Then, to find the pose as well as the intrinsic parameters, the following steps can be applied:

- 1) Let M_1 be the first 3×3 matrix of P
- 2) Normalize P by multiplying all elements with $\text{sign}(\det(M_1))$
- 3) Compute C by SVD
- 4) Compute the camera center C by SVD (Since $PC=0$, the SVD algorithm could be used directly)

To estimate C using the SVD, you need to use the following code:

```
>> [U,S,V] = svd(P);  
>> C = V(1:3, size(V,2)) / V(size(V,1), size(V,2));
```

- 5) Apply an RQ factorization to M_1 using the provided MATLAB function such as $M_1 = \tilde{K}\tilde{R}$ such that $\tilde{K}$ an upper triangular matrix and $\tilde{R}$ an orthogonal matrix.
- 6) Define $D = \text{diag}(\text{sign}(K_{11}), \text{sign}(K_{22}), \text{sign}(K_{33}))$
- 7) Compute K , R and t such as $K = \tilde{K}D$, $R = D\tilde{R}$ and $t = -RC$.

- Based on the described steps, compute the intrinsic and extrinsic matrices using MATLAB. In the report, provide the used code and outputs. Report the obtained matrix pose T .

Useful MATLAB commands:

```
>>sign(A)  
% returns the sign of a real a (1 if positive, -1 if  
negative)  
>> det(A)  
% returns the determinant of a matrix A  
>> diag(vec)  
% returns a diagonal matrix with the values of vec on the  
diagonal
```